

삼성청년 SW·AI아카데미

B형 특강

<알림>

본 강의는 삼성청년SW·AI아카데미의 콘텐츠로
보안서약서에 의거하여
강의 내용을 어떠한 사유로도 임의로 복사, 촬영,
녹음, 복제, 보관, 전송하거나
허가 받지 않은 저장매체를
이용한 보관, 제3자에게 누설, 공개,
또는 사용하는 등의 행위를 금합니다.

Pro 연습 문제. 가구 공장

가구 공장 - 문제 이해

고객이 원하는 대로 가구를 만들어 주는 공장이 있다.
이 공장은 오직 가구에 사용되는 합판의 가격만 받고 가구를 제작해 준다.

공장은 2개의 원목 창고를 가지고 있다.
가구에 사용될 합판을 만들기 위해서는 상판, 중판, 하판이 될 원목을 각각 1개씩 선택하여야 한다.
고객이 지불하는 합판의 가격은 각각의 원목들의 가격의 합과 같다.
만약, 두 창고의 원목을 섞어 합판을 만들어야 한다면, 합판의 가격에 운송료를 더하여 지불한다.

또한, 원목은 각각 최대한 버틸 수 있는 무게인 내하중이 존재한다.
합판의 내하중은 3종류의 원목의 내하중의 최소값과 같다.

공장은 언제나 고객의 예산 내에서 가장 내하중이 높은 가구를 만들어 제작을 주문한 고객에게 판매한다.
만약, 같은 내하중을 낼 수 있는 원목의 조합이 여러가지라면, 그 중 가격이 가장 저렴한 합판으로 가구를 제작하여 판매한다.

Warehouse 0			Warehouse 1		
Type	Price	Load Capacity	Type	Price	Load Capacity
0	100	200	0	150	150
0	200	100	0	250	300
1	220	120	1	120	100
1	330	160	1	450	200
2	77	50	2	300	200
2	88	150	2	130	70

[table 1]

가구 공장 - 문제 이해

각 창고에 보관된 원목의 목록이 [table1]과 같고 운송료가 50이라고 할 때, 예산이 400인 고객이 가구 주문 제작을 의뢰 했다고 하자.

Warehouse 0		
Type	Price	Load Capacity
0	100	200
0	200	100
1	220	120
1	330	160
2	77	50
2	88	150

[table 2]

창고 0의 원목만 고르면 [table 2]와 같은 조합의 합판을 만들 수 있다. 이 때의 가격은 $100 + 220 + 77 = 397$ 이고, 내하중은 200, 120, 50 중 최소 값인 50이 된다.

Warehouse 1		
Type	Price	Load Capacity
0	150	150
0	250	300
1	120	100
1	450	200
2	300	200
2	130	70

[table 3]

창고 1의 원목만 고르면 [table 3]와 같은 조합의 합판을 만들 수 있다. 이 때의 가격은 $150 + 120 + 130 = 400$ 이고, 내하중은 150, 100, 70 중 최소 값인 70이 된다.

가구 공장 - 문제 이해

Warehouse 0		
Type	Price	Load Capacity
0	100	200
0	200	100
1	220	120
1	330	160
2	77	50
2	88	150

Warehouse 1		
Type	Price	Load Capacity
0	150	150
0	250	300
1	120	100
1	450	200
2	300	200
2	130	70

[table 4]

창고 0, 1에서 모두 원목을 고르면 [table 4] 과 같이 합판 가격 $100 + 120 + 88 = 308$ 에 운송료 50을 더해 358의 비용으로
내하중은 200, 100, 150 중 최소인 100의 내하중을 갖는 가구를 만들 수 있고, 이 조합이 주어진 예산 내에서 최대의 내하중을 갖는 합판으로 만든 가구이다.
이와 같이 각 창고에 보관 중인 원목의 목록을 관리하고, 고객의 예산을 받아 가구 제작 주문을 처리하는 프로그램을 구현하라.

가구 공장 - API

```
public void init(int mCharge)
```

각 테스트 케이스의 처음에 호출되며, 창고 간 운송료를 알려준다.

- 기존의 원목 목록은 모두 초기화된다.

Parameters

- mCharge : 창고 간 운송료 ($1 \leq mCharge \leq 100,000$)

가구 공장 - API

```
public int stock(int mType, int mPrice, int mLoadCapacity, int mPosition)
```

부품 목록에 새로운 원목을 추가한다.

- 종류와 가격, 성능이 모두 같은 입력이 testcase 내에서 중복되어 들어오지 않는다.

Parameters

- mType : 원목 종류 ($0 \leq mType \leq 2$)
- mPrice : 원목 가격 ($1 \leq mPrice \leq 100,000$)
- mLoadCapacity: 원목 내하중 ($1 \leq mLoadCapacity \leq 1,000,000$)
- mPosition : 원목이 있는 창고 번호 ($0 \leq mPosition \leq 1$)

Returns

- 입고된 창고에 동일한 type인 원목의 가짓수를 반환한다.

가구 공장 - API

```
public Main.Result order(int mBudget)
```

고객의 예산이 mBudget일 때 예산 안에서 최대 내하중을 가지는 원목의 조합을 선택한다.

- 같은 내하중이면 가격이 가장 낮은 조합을 선택한다.
- 가구가 출고되면 사용한 원목을 즉각 채워 넣으므로 품절되지 않는다.

Parameters

- mBudget: 소비자의 예산 ($1 \leq \text{mBudget} \leq 500,000$)

Returns

- 주어진 예산으로 가구 제작이 가능할 경우, Result 구조체의 mLoadCapacity에 가구의 내하중을, mPrice에 가구의 가격을 반환한다.
- 주어진 예산으로 가구 제작이 불가능할 경우, mLoadCapacity와 mPrice 모두 0을 반환한다.

[제약사항]

1. 각 테스트 케이스 시작 시 init() 함수가 호출된다.
2. 각 테스트 케이스에서 stock() 함수의 호출 횟수는 4,000 이하이다.
3. 각 테스트 케이스에서 order() 함수의 호출 횟수는 400 이하이다.

가구 공장 - 예시

order	Function	return	table
1	init(50)		
2	stock(0,100,200,0)	1	
3	stock(0,200,100,0)	2	
4	stock(1,220,120,0)	1	
5	stock(1,330,160,0)	2	
6	stock(2,77,50,0)	1	
7	stock(1,88,150,0)	2	
8	stock(0,150,150,1)	1	
9	stock(0,250,300,1)	2	
10	stock(1,120,100,1)	1	
11	stock(1,450,200,1)	2	
12	stock(2,300,200,1)	1	
13	stock(2,130,70,1)	2	
14	order(400)	mPrice = 358, mLoadCapacity = 100	[table 4]
15	order(450)	mPrice = 408, mLoadCapacity = 120	[table 5]
16	order(346)	mPrice = 0, mLoadCapacity = 0	

Warehouse 0		
Type	Price	Load Capacity
0	100	200
0	200	100
1	220	120
1	330	160
2	77	50
2	88	150

[table 4]

Warehouse 1		
Type	Price	Load Capacity
0	150	150
0	250	300
1	120	100
1	450	200
2	300	200
2	130	70

Warehouse 0		
Type	Price	Load Capacity
0	100	200
0	200	100
1	220	120
1	330	160
2	77	50
2	88	150

[table 5]

설계하기

삼성청년SW·AI아카데미

❖ Parametric Search

- PS 문제임을 파악할 수 있는 요소들
 - 1) 특정 조건을 충족하는 값중의 min 또는 max를 찾는 것.
 - 예산 내로 구매할 수 있는 가구 중 내하중이 가장 높은 것.



이 금액(mBudget)으로 이 내하중 (mLoadCapacity)의 가구를 제작할 수 있을까?



의 한계점 / 최적해를 binary search (parametric search)로 빠르게 탐색

❖ 원목이 추가된다.

- 반환해야 하는 값은 mPosition 창고에 존재하는 mType의 원목의 개수이다
 - mType, mPosition 별로 분리해서 관리해야 한다.
- 단순히 mPosition 창고에 mType 원목에 대해 이런 (mPrice, mLoadCapacity)의 원목이 추가된다고 기록하면 된다.
- 시간복잡도 : $O(1)$

order() 설계

❖ order()

- Parametric Search logic
 - 주어진 mBudget으로 제작할 수 있는 최대 내하중의 가구를 제작한다.
 - **left** : 최소 내하중
 - **right** : 최소 내하중
 - **while(left <= right)**
 - mid : mBudget이면 이 mid 내하중(이상)의 가구를 제작할 수 있다!
 - test(mid, mBudget)

어떻게 test() 를 해야 할까?

- **test() 성공** : 더 강한 내하중을 가지는 가구를 제작할 수 있을까?
- **test() 실패** : 더 낮은 내하중을 가지는 가구를 제작해보자.

❖ test(mid, mBudget)

- mBudget으로 mid 내하중 이상의 가구를 제작할 수 있는지 확인해야 한다.
- 그리고 만들 수 있었다면, 그 제작된 가구의 값을 확인할 수 있어야 한다.

❖ 가구를 만들 수 있는 방법

- 창고 0에서 모든 원목을 사용한다.
- 창고 1에서 모든 원목을 사용한다.
- 창고 0, 1에 존재하는 원목을 섞고, 운송료를 추가한다.

❖ 원목의 내하중을 결정짓는 요소

- 사용한 원목 중 최하 내하중이 가구의 최종 내하중이 된다.

❖ 그렇다면, 조합해볼까?

- 원목의 개수는 최대 4,000개이다. (stock()의 호출 횟수)
- 공평하게 나눠졌다고 가정하면, 각 창고의 각 mType의 원목은 약 666개씩 존재할것이다.

창고 0	창고 1
원목 0 (666)	원목 0 (666)
원목 1 (666)	원목 1 (666)
원목 2 (666)	원목 2 (666)

- 단순 조합 :

```
for(원목 0 : 창고[0][0])  
  for (원목 1 : 창고[0][1])  
    for (원목 2 : 창고[0][2])  
      if(원목0 + 원목1 + 원목2 >= target 내하중)  
        가격 갱신
```

- 한 창고에 대한 조합만 만들어봐도 $O(666^3) = 295,408,296$ (TLE)

test() 설계

❖ 제작된 가구의 내하중 = 사용된 원목 중 가장 낮은 내하중

- 즉, 3개의 원목 모두 내 target 내하중 이상이어야한다.
- target 내하중 이상인 원목 중 가장 싼 원목 하나씩만 선정하자.
- 시간복잡도 : $O(\text{전체 원목의 수}) = 4,000$

창고 0	창고 1
원목 0 (666)	원목 0 (666)
원목 1 (666)	원목 1 (666)
원목 2 (666)	원목 2 (666)

Target내하중 이상인 원목중 제일 싼것 체크



창고 0	창고 1
원목 0 (1)	원목 0 (1)
원목 1 (1)	원목 1 (1)
원목 2 (1)	원목 2 (1)

어떤 조합을 하더라도
target 내하중 충족 가능한 후보군

❖ 후보군을 추려냈으면, 조합을 $O(1)$ 로 해볼 수 있다.

- 창고 0 only : $A0.price + A1.price + A2.price$
- 창고 1 only : $B0.price + B1.price + B2.price$
- 창고 0 & 1 : $\min(A0.price, B0.price) + \min(...) + \min(...) + \text{운송료}$

창고 0	창고 1
원목 0 (A0)	원목 0 (B0)
원목 1 (A1)	원목 1 (B1)
원목 2 (A2)	원목 2 (B2)

- 이렇게 해서 만들어낸 조합 중 가장 싼 것 = mid 내하중 이상의 가구 중 제일 저렴한 제작
 - 이 가구를 mBudget내로 해결이 가능하다면 → test 성공 → 가격 return

최종 시간복잡도

❖ 시간복잡도

- **Parametric Search Logic**

- $O(\log \text{최대 내하중}) = O(\log(1,000,000)) = \sim 20$

- **test() :**

- 후보군 선정 : $O(4,000)$
 - 조합 : $O(1)$

- **testcase의 개수**

- $400 \times (\sim 20 \times 4,000) = 32,000,000$